

AIE683 Machine Learning

Week 7: Model Evaluation, Calibration & Imbalanced Data

Dr. Hakan Emekci

Hacettepe University, Graduate School of Informatics

Screening for 1 in 40

A screening program sees thousands of mammograms. About **2%** contain a finding (micro-calcifications) that needs a closer look.

- A model that says "normal" for every image is **97.7% accurate**
- Missing a finding costs far more than an extra radiologist review
- The radiologist wants a **probability**, not just a yes/no

Plan for today

- Part 1 (≈ 80 min): metrics for binary and multi-class problems; decision thresholds
 - Try it 1: predict which model wins
 - Try it 2: set the threshold for the clinic
- Break
- Part 2 (≈ 80 min): calibration; imbalanced data (threshold first, class weights, resampling as an experiment)
 - Try it 3: spot the leak
- Wrap-up and lab (lab.ipynb)

Metrics for Binary Classification

Review: confusion matrix

actual negative	True Negative	False Positive
	False Negative	True Positive
actual positive		
	predicted negative	predicted positive

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

Positive & Negative are arbitrary

**Though often the minority class is considered
positive**

#

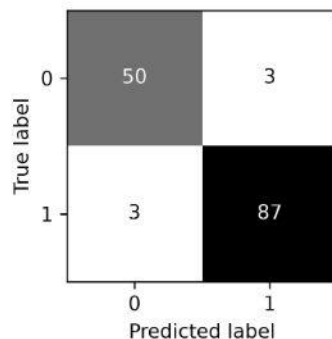
```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
data = load_breast_cancer()

X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, stratify=data.target, random_state=0)

lr = make_pipeline(StandardScaler(), LogisticRegression()).fit(X_train, y_train)
y_pred = lr.predict(X_test)

print(confusion_matrix(y_test, y_pred))
print(lr.score(X_test, y_test))
ConfusionMatrixDisplay.from_estimator(lr, X_test, y_test, cmap='gray_r')
```

```
[[50  3]
 [ 3 87]]
0.958
```

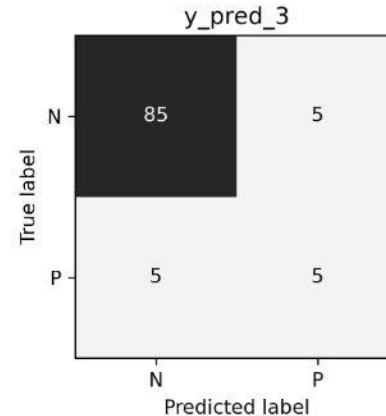
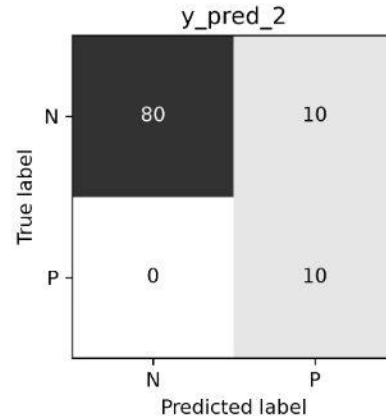
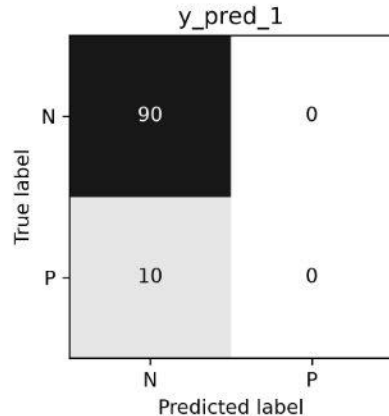


Problems with Accuracy

Data with 90% negatives:

```
from sklearn.metrics import accuracy_score
for y_pred in [y_pred_1, y_pred_2, y_pred_3]:
    print(accuracy_score(y_true, y_pred))
```

0.9
0.9
0.9



Precision, Recall, f-score

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Positive Predictive Value (PPV)

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Sensitivity, coverage, true positive rate.

$$F = 2 \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

Harmonic mean of precision and recall

Normalizing the confusion matrix

```
confusion_matrix(y_true, y_pred)
```

actual negative	True Negative	False Positive
	False Negative	True Positive
actual positive	False Negative	True Positive
	predicted negative	predicted positive

```
confusion_matrix(y_true, y_pred,  
normalize='true')
```

Normalized by true condition

actual negative	True Negative Rate Specificity	False Positive Rate False Positive Rate Fall-out
	False Negative Rate Miss rate	True Positive Rate True Positive Rate Recall Sensitivity
actual positive	False Negative Rate Miss rate	True Positive Rate True Positive Rate Recall Sensitivity
	predicted negative	predicted positive

```
confusion_matrix(y_true, y_pred,  
normalize='pred')
```

Normalized by predicted condition

actual negative	Negative Predictive Value	False Discovery Rate
	False Omission Rate	Precision Positive Predictive Value
actual positive	False Omission Rate	Precision Positive Predictive Value
	predicted negative	predicted positive

```
classification_report(y_true, y_pred)
```

y_pred_1

True label \ Predicted label	N	P
N	90	0
P	10	0

	precision	recall	f1-score	support
0	0.90	1.00	0.95	90
1	0.00	0.00	0.00	10
accuracy			0.90	100
macro avg	0.45	0.50	0.47	100
weighted avg	0.81	0.90	0.85	100

y_pred_2

True label \ Predicted label	N	P
N	80	10
P	0	10

	precision	recall	f1-score	support
0	1.00	0.89	0.94	90
1	0.50	1.00	0.67	10
accuracy			0.90	100
macro avg	0.75	0.94	0.80	100
weighted avg	0.95	0.90	0.91	100

y_pred_3

True label \ Predicted label	N	P
N	85	5
P	5	5

	precision	recall	f1-score	support
0	0.94	0.94	0.94	90
1	0.50	0.50	0.50	10
accuracy			0.90	100
macro avg	0.72	0.72	0.72	100
weighted avg	0.90	0.90	0.90	100

Averaging strategies

$$\text{macro } \frac{1}{|L|} \sum_{l \in L} R(y_l, \hat{y}_l)$$

$$\text{weighted } \frac{1}{n} \sum_{l \in L} n_l R(y_l, \hat{y}_l)$$

```
print("Weighted average: ", recall_score(y_true, y_pred_1, average="weighted"))  
print("Macro average: ", recall_score(y_true, y_pred_1, average="macro"))
```

Weighted average: 0.90
Macro average: 0.50

Balanced Accuracy

```
balanced_accuracy_score(y_t, y_p) == recall_score(y_t, y_p, average='macro')
```

$$\text{balanced accuracy} = \frac{1}{2} \left(\frac{\text{TP}}{\text{TP} + \text{FN}} + \frac{\text{TN}}{\text{TN} + \text{FP}} \right)$$

- Always 0.5 for chance predictions
- Equal to accuracy for balanced datasets
- Related single-number alternative: Matthews correlation coefficient (`matthews_corrcoef`)

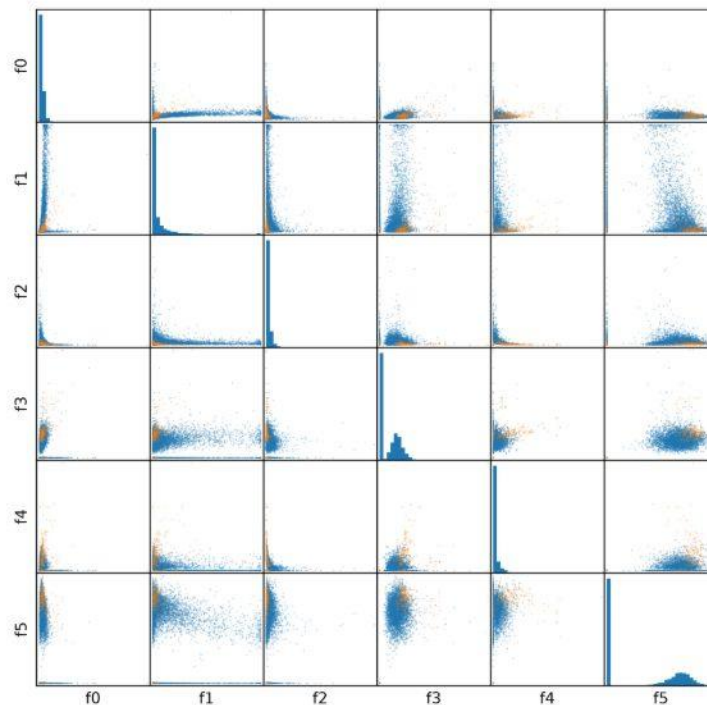
Mammography Data

```
from sklearn.datasets import fetch_openml
# mammography https://www.openml.org/d/310
data = fetch_openml('mammography', version=1, as_frame=True)
X, y = data.data, data.target
X.shape
```

(11183, 6)

```
y.value_counts()
```

```
-1    10923
1       260
```



```
# make y boolean
# this allows sklearn to determine the positive class more easily
X_train, X_test, y_train, y_test = train_test_split(X, y == '1', random_state=0)
```

```

svc = make_pipeline(StandardScaler(),
                    SVC(C=100, gamma=0.1))
svc.fit(X_train, y_train)
svc.score(X_test, y_test)

```

0.986

	precision	recall	f1-score	support
False	0.99	1.00	0.99	2732
True	0.81	0.53	0.64	64
accuracy			0.99	2796
macro avg	0.90	0.76	0.82	2796
weighted avg	0.98	0.99	0.99	2796

```

rf = RandomForestClassifier(random_state=0)
rf.fit(X_train, y_train)
rf.score(X_test, y_test)

```

0.988

	precision	recall	f1-score	support
False	0.99	1.00	0.99	2732
True	0.89	0.53	0.67	64
accuracy			0.99	2796
macro avg	0.94	0.76	0.83	2796
weighted avg	0.99	0.99	0.99	2796

Goal setting!

- What do I want? What do I care about?
- Can I assign costs to the confusion matrix?
- What guarantees do we want to give?

Changing Thresholds

```
y_pred = rf.predict(X_test)
print(classification_report(y_test, y_pred))
```

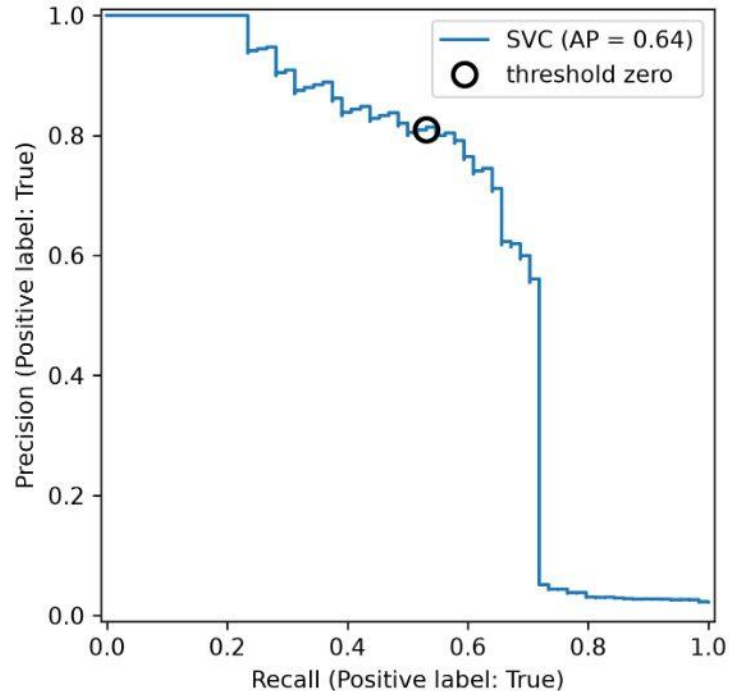
	precision	recall	f1-score	support
False	0.99	1.00	0.99	2732
True	0.89	0.53	0.67	64
accuracy			0.99	2796
macro avg	0.94	0.76	0.83	2796
weighted avg	0.99	0.99	0.99	2796

```
y_pred = rf.predict_proba(X_test)[:, 1] > .30
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
False	0.99	0.99	0.99	2732
True	0.75	0.66	0.70	64
accuracy			0.99	2796
macro avg	0.87	0.83	0.85	2796
weighted avg	0.99	0.99	0.99	2796

Precision-Recall curve

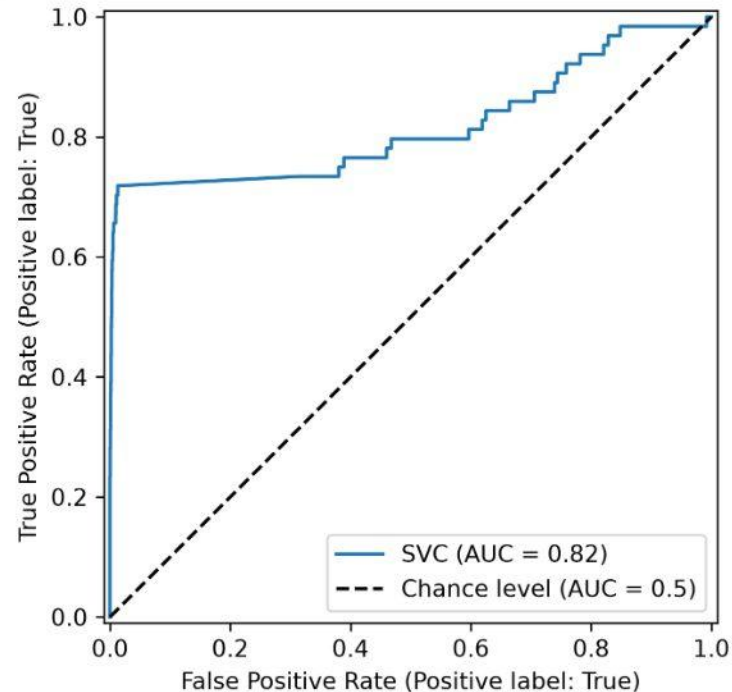
```
from sklearn.metrics import PrecisionRecallDisplay
svc = make_pipeline(StandardScaler(), SVC(C=100, gamma=0.1))
svc.fit(X_train, y_train)
PrecisionRecallDisplay.from_estimator(svc, X_test, y_test, name='SVC')
```



ROC Curve

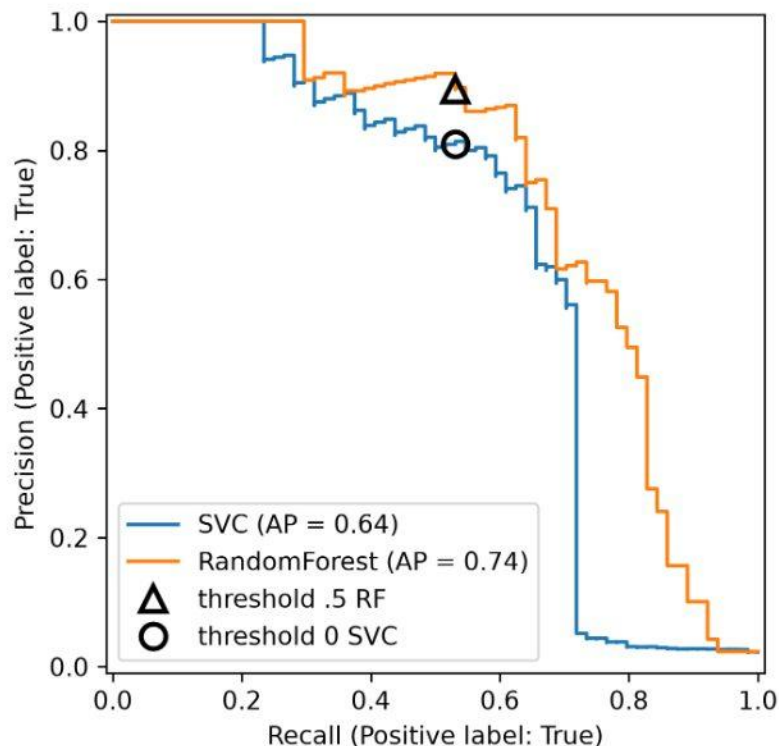
(Receiver Operating Characteristic)

```
from sklearn.metrics import RocCurveDisplay  
RocCurveDisplay.from_estimator(svc, X_test, y_test, name='SVC',  
                              plot_chance_level=True)
```



Comparing RF and SVC

```
fig, ax = plt.subplots()
pr_svc.plot(ax=ax, name='SVC')          # reuse the computed display
PrecisionRecallDisplay.from_estimator(rf, X_test, y_test, ax=ax, name='RandomForest')
```



Average Precision

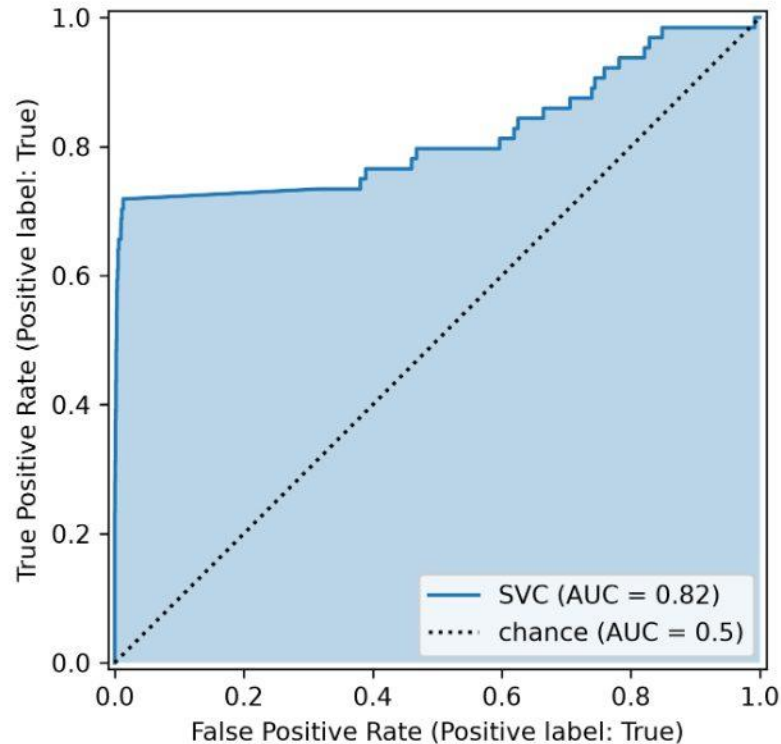
Precision at threshold k

$$\text{AveP} = \sum_{k=1}^n P(k) \Delta r(k)$$

Change in recall between k and $k-1$

Sum over data points, ranked by decision function

Area Under ROC Curve (aka AUC)



- Always .5 for random/constant prediction
- = probability that a random positive is ranked above a random negative

Keep in mind: ranking vs predictions

```
dec = svc.decision_function(X_test)
np.all((dec > 0) == svc.predict(X_test))
```

True

```
print(f1_score(y_test, dec > 0))
print(average_precision_score(y_test, dec))
```

0.642

0.635

```
dec_new = dec - 10
print(f1_score(y_test, dec_new > 0))
print(average_precision_score(y_test, dec_new))
```

0.0

0.635

Threshold and ranking metrics

```
from sklearn.metrics import f1_score
f1_rf = f1_score(y_test, rf.predict(X_test))
print(f"f1_score of random forest: {f1_rf:.3f}")
f1_svc = f1_score(y_test, svc.predict(X_test))
print(f"f1_score of svc: {f1_svc:.3f}")
```

f1_score of random forest: 0.667
f1_score of svc: 0.642

```
from sklearn.metrics import balanced_accuracy_score
ba_rf = balanced_accuracy_score(y_test, rf.predict(X_test))
print(f"Balanced accuracy of random forest: {ba_rf:.3f}")
ba_svc = balanced_accuracy_score(y_test, svc.predict(X_test))
print(f"Balanced accuracy of svc: {ba_svc:.3f}")
```

Balanced accuracy of random forest: 0.765
Balanced accuracy of svc: 0.764

```
from sklearn.metrics import average_precision_score
ap_rf = average_precision_score(
    y_test, rf.predict_proba(X_test)[: , 1])
print(f"Average precision of random forest: {ap_rf:.3f}")
ap_svc = average_precision_score(
    y_test, svc.decision_function(X_test))
print(f"Average precision of svc: {ap_svc:.3f}")
```

Average precision of random forest: 0.737
Average precision of svc: 0.635

```
from sklearn.metrics import roc_auc_score
auc_rf = roc_auc_score(y_test, rf.predict_proba(X_test)[: , 1])
print(f"Area under ROC curve of random forest: {auc_rf:.3f}")
auc_svc = roc_auc_score(y_test, svc.decision_function(X_test))
print(f"Area under ROC curve of svc: {auc_svc:.3f}")
```

Area under ROC curve of random forest: 0.936
Area under ROC curve of svc: 0.817

Summary of metrics for binary classification

Threshold-based:

- (balanced) accuracy
- precision, recall, F1 (or F-beta)
- expected cost from a cost matrix

Ranking:

- average precision
- ROC AUC

Probabilistic (Part 2):

- log loss, Brier score

Try it 1: predict which model wins (10 min)

Lab section 2. Three models on the mammography data, 5-fold CV:

```
LogisticRegression, RandomForestClassifier,  
RandomForestClassifier(class_weight="balanced")
```

1. **Before running:** write down the winner on accuracy, balanced accuracy, average precision, ROC AUC
2. Complete the TODO and run it
3. Which metric changes the ranking most? Why?

Decision thresholds

The threshold is part of the model

```
from sklearn.model_selection import FixedThresholdClassifier, TunedThresholdClassifier

# set it by hand (e.g. a threshold agreed with domain experts)
rf_03 = FixedThresholdClassifier(RandomForestClassifier(random_state=0), threshold=0.414)
rf_03.fit(X_train, y_train)

# or tune it with cross-validation on the training data
tuned_f1 = TunedThresholdClassifierCV(RandomForestClassifier(random_state=0),
                                     scoring="f1", cv=5)
tuned_f1.fit(X_train, y_train)
print(f"{tuned_f1.best_threshold_:.3f}")
print(f1_score(y_test, rf.predict(X_test)), f1_score(y_test, tuned_f1.predict(X_test)))
```

0.414

0.667 0.704

- Both are ordinary estimators: they work in pipelines, `cross_validate`, `GridSearchCV`
- Available since scikit-learn 1.5

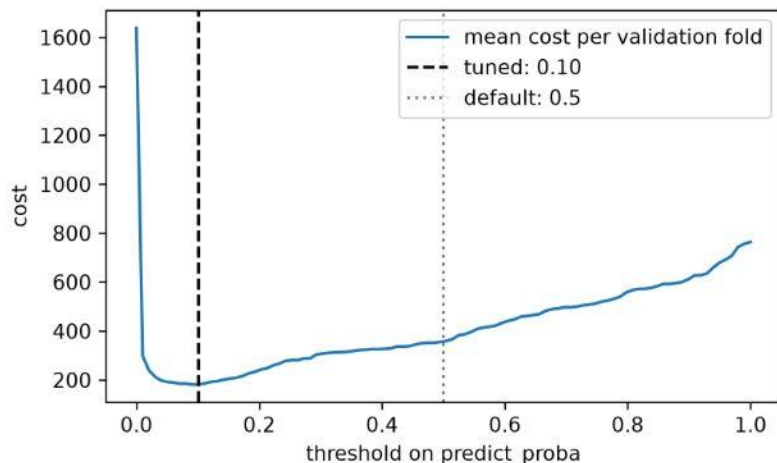
Cost-sensitive threshold

```
COST_FP, COST_FN = 1, 20      # missed finding = 20 extra reviews
def neg_cost(y_true, y_pred):
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred, labels=[False, True]).ravel()
    return -(COST_FP * fp + COST_FN * fn)

tuned_cost = TunnedThresholdClassifierCV(RandomForestClassifier(random_state=0),
                                         scoring=make_scorer(neg_cost), cv=5,
                                         store_cv_results=True).fit(X_train, y_train)
```

cost-optimal threshold: 0.101

test cost default 0.5: 604 tuned for F1: 526 tuned for cost: 285



Thresholds: what goes wrong in practice

- **Tuning on the test set:** the reported cost is optimistic. Tune inside CV (or on a validation split), report on the test set
- **Prevalence shift:** a threshold tuned at 2% positives is wrong at 5%; precision-based thresholds are most affected
- **Uncalibrated scores:** "0.3" means nothing by itself; the optimal cut-off depends on the model
- **Multi-class:** no single threshold; use per-class costs on calibrated probabilities (decision theory), or reduce to one-vs-rest decisions

Try it 2: set the threshold for the clinic (15 min)

Lab section 4. Costs: missed finding **50**, false alarm **1**.

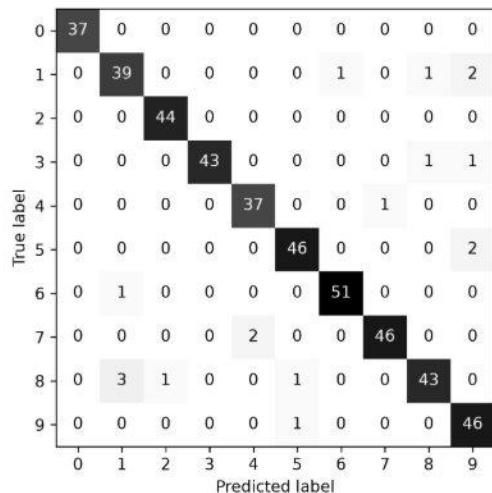
1. Any classifier you like; choose its threshold **using only training data**
2. Report `total_cost(y_test, y_pred, cost_fp=1, cost_fn=50)` on the test set
3. Lowest test cost wins; tuning on the test set disqualifies

Confusion Matrix

```
from sklearn.datasets import load_digits
from sklearn.metrics import accuracy_score

digits = load_digits()
# data is between 0 and 16
X_train, X_test, y_train, y_test = train_test_split(
    digits.data / 16., digits.target, random_state=0)
lr = LogisticRegression(max_iter=1000).fit(X_train, y_train)
pred = lr.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, pred):.3f}")
ConfusionMatrixDisplay.from_estimator(lr, X_test, y_test,
                                     cmap='gray_r')
```

Accuracy: 0.960



```
print(classification_report(y_test, pred))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	37
1	0.91	0.91	0.91	43
2	0.98	1.00	0.99	44
3	1.00	0.96	0.98	45
4	0.95	0.97	0.96	38
5	0.96	0.96	0.96	48
6	0.98	0.98	0.98	52
7	0.98	0.96	0.97	48
8	0.96	0.90	0.92	48
9	0.90	0.98	0.94	47
accuracy			0.96	450
macro avg	0.96	0.96	0.96	450
weighted avg	0.96	0.96	0.96	450

Summary of metrics for multiclass classification

Threshold-based:

- accuracy
- precision, recall, f1 (macro average, weighted)

Ranking:

- OVR ROC AUC
- OVO ROC AUC

Probabilistic:

- log loss (cross-entropy)

Picking metrics

- Accuracy rarely what you want
- Problems are rarely balanced
- Find the right criterion for the task
- OR pick a substitute, but at least think about it
- Emphasis on recall or precision?
- Which classes are the important ones?

Using metrics in cross-validation

```
cancer = load_breast_cancer()
X, y = cancer.data, cancer.target

# default scoring for classification is accuracy
rf = RandomForestClassifier(random_state=0)
print("default scoring ", cross_val_score(rf, X, y))

# providing scoring="accuracy" doesn't change the results
explicit_accuracy = cross_val_score(rf, X, y, scoring="accuracy")
print("explicit accuracy scoring ", explicit_accuracy)

ap = cross_val_score(rf, X, y, scoring="average_precision")
print("average precision", ap)
```

```
default scoring [0.93  0.947 0.991 0.974 0.973]
explicit accuracy scoring [0.93  0.947 0.991 0.974 0.973]
average precision [0.992 0.973 0.999 0.995 0.999]
```

Multiple Metrics

```
from sklearn.model_selection import cross_validate
res = cross_validate(RandomForestClassifier(random_state=0), X, y,
                    scoring=["accuracy", "average_precision", "recall_macro"],
                    return_train_score=True, cv=5)
pd.DataFrame(res)
```

	fit_time	score_time	test_accuracy	train_accuracy	test_average_precision	train_average_precision	test_recall_macro	train_recall_macro
0	0.161	0.012	0.930	1.0	0.992	1.0	0.925	1.0
1	0.107	0.008	0.947	1.0	0.973	1.0	0.935	1.0
2	0.099	0.010	0.991	1.0	0.999	1.0	0.993	1.0
3	0.157	0.007	0.974	1.0	0.995	1.0	0.969	1.0
4	0.097	0.012	0.973	1.0	0.999	1.0	0.974	1.0

Break

Part 2: Calibration, Imbalanced Data

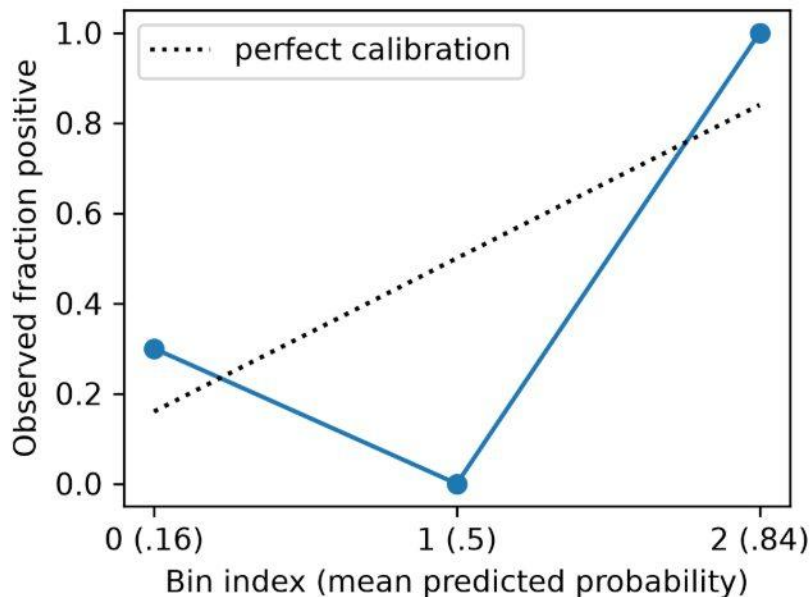
Calibration

Niculescu-Mizil & Caruana, Predicting good probabilities with supervised learning, ICML 2005

- Probabilities can be much more informative than labels:
- “The model predicted you don’t have cancer” vs “The model predicted you’re 40% likely to have cancer”

Calibration curve (Reliability diagram)

$\hat{p}(y)$	y		$\hat{p}(y)$	y	bin
0.9	1	sort →	0.9	1	}
0.4	0		0.8	1	
0.3	1		0.6	0	}
0.6	0		0.4	0	
0.8	1		0.3	1	}
0.2	0		0.3	0	
0.3	0		0.2	0	



calibration_curve with sklearn

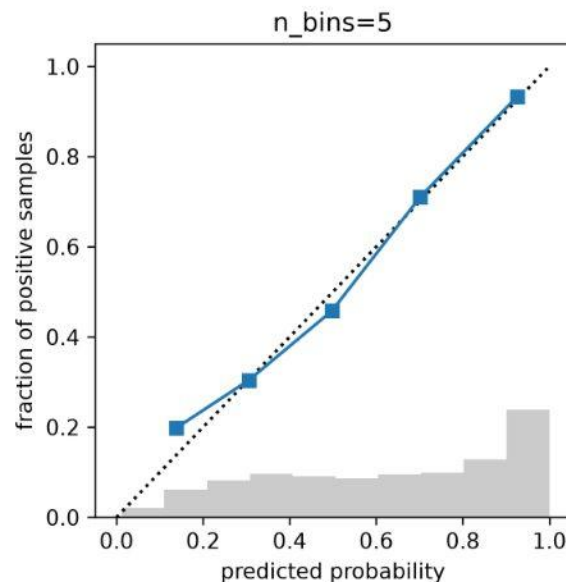
Using subsample of coverytype dataset

```
from sklearn.linear_model import LogisticRegressionCV
print(X_train.shape)
print(np.bincount(y_train))
lr = LogisticRegressionCV(max_iter=1000, scoring="neg_log_loss",
                          l1_ratios=(0,)).fit(X_train, y_train)
(52292, 54)
[19036 33256]
```

```
lr.C_
2.783
```

```
print(lr.predict_proba(X_test)[:10])
print(y_test[:10])
[[0.682 0.318]
 [0.049 0.951]
 [0.705 0.295]
 [0.537 0.463]
 [0.819 0.181]
 [0.    1.   ]
 [0.795 0.205]
 [0.676 0.324]
 [0.728 0.272]
 [0.596 0.404]]
[0 1 0 1 1 1 0 0 0 1]
```

```
from sklearn.calibration import calibration_curve
probs = lr.predict_proba(X_test)[: , 1]
prob_true, prob_pred = calibration_curve(y_test, probs, n_bins=5)
print(prob_true)
print(prob_pred)
[0.198 0.304 0.458 0.71  0.933]
[0.138 0.306 0.498 0.702 0.926]
```



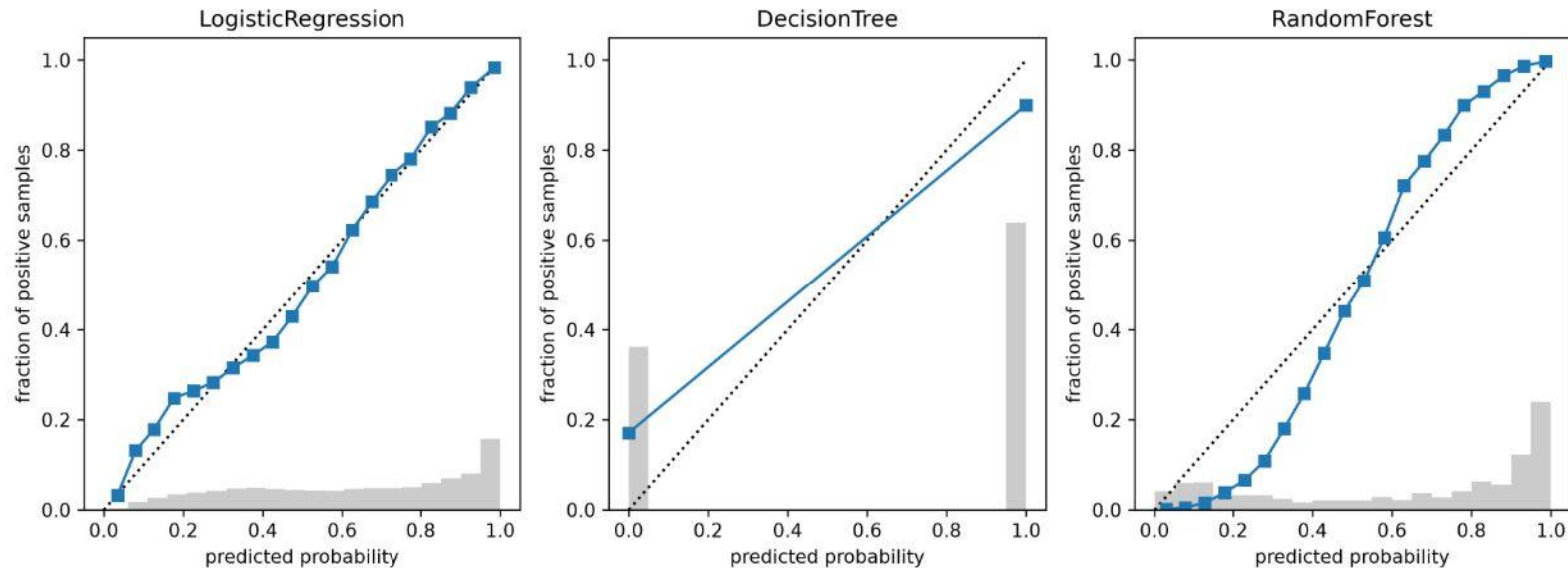
CalibrationDisplay

```
from sklearn.calibration import CalibrationDisplay

fig, ax = plt.subplots()
CalibrationDisplay.from_estimator(lr, X_test, y_test, n_bins=10, name="LogReg", ax=ax)
CalibrationDisplay.from_predictions(y_test, rf_probs, n_bins=10, name="RF", ax=ax)
# strategy="quantile" puts the same number of samples in each bin
```

- Same pattern as the other `*Display` classes
- Keeps `prob_true`, `prob_pred`, `y_prob` for further use

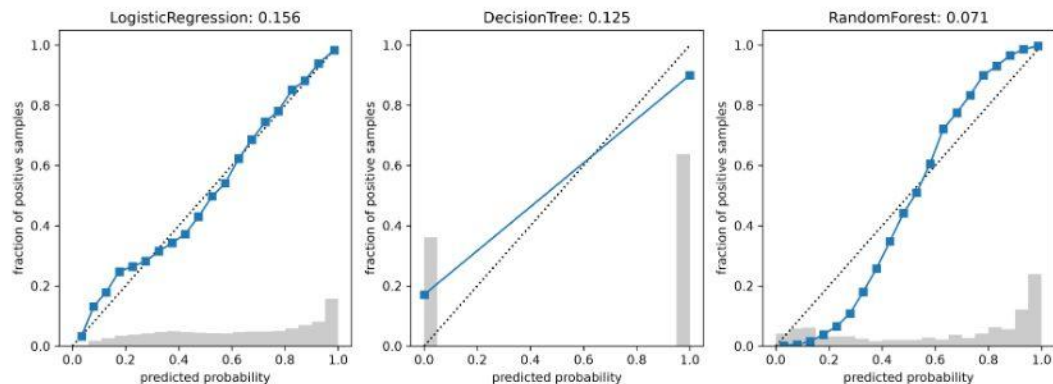
Comparing Models



Brier Score (for binary classification)

- “mean squared error of probability estimate”

$$BS = \frac{1}{n} \sum_{i=1}^n (\hat{p}(y_i = 1 \mid x_i) - y_i)^2$$



	Brier	log loss	ROC AUC
LogisticRegression	0.156	0.467	0.841
DecisionTree	0.125	1.733	0.864
RandomForest	0.071	0.247	0.970

Fixing it: Calibrating a classifier

- Build another model, mapping classifier probabilities to better probabilities!
- 1d model! (or more for multi-class)

$$f_{\text{calib}}(s(x)) \approx p(y = 1 \mid x)$$

- $s(x)$ is the score given by the model
- Can also work with models that don't even provide probabilities!
- Need a model for f_{calib} , and need to decide what data to train it on.
 - Train on training set → overfit
 - Train using a hold-out set or cross-validation → unbiased, costs data or time

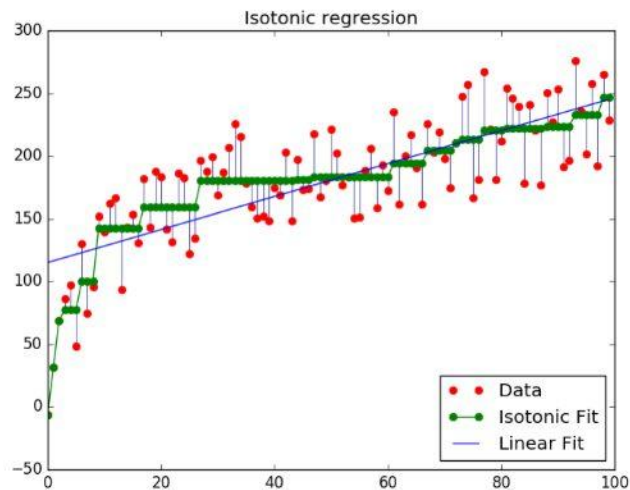
Platt Scaling

- Use a logistic sigmoid for f_{calib}
- Basically learning a 1d logistic regression
- (+ some tricks)
- Works well for SVMs

$$f_{\text{platt}} = \frac{1}{1 + \exp(-ws(x) - b)}$$

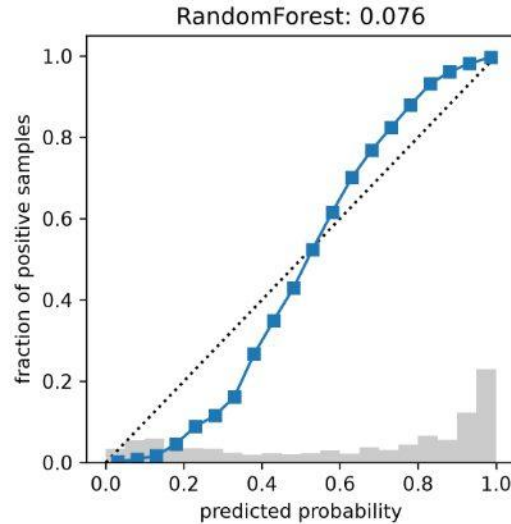
Isotonic Regression

- Very flexible way to specify f_{calib}
- Learns arbitrary monotonically increasing step-functions in 1d.
- Groups data into constant parts, steps in between.
- Optimum monotone function on training data (wrt mse).



CalibratedClassifierCV

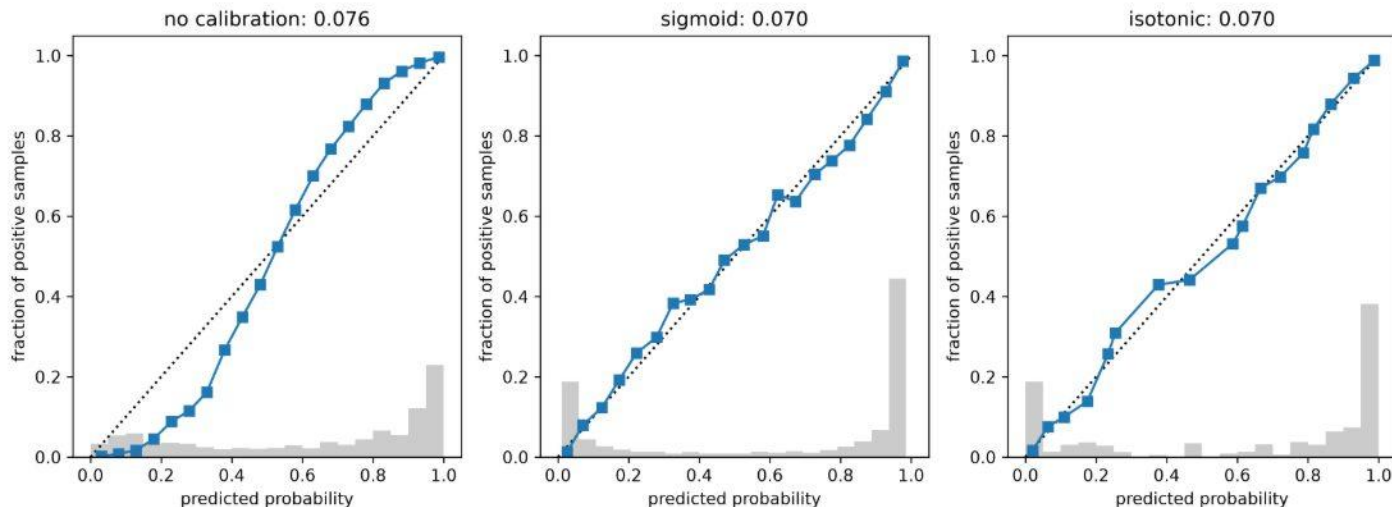
```
from sklearn.calibration import CalibratedClassifierCV
X_train_sub, X_val, y_train_sub, y_val = train_test_split(X_train, y_train,
                                                          stratify=y_train, random_state=0)
rf = RandomForestClassifier(random_state=0).fit(X_train_sub, y_train_sub)
scores = rf.predict_proba(X_test)[: , 1]
CalibrationDisplay.from_predictions(y_test, scores, n_bins=20)
```



Calibration on Random Forest

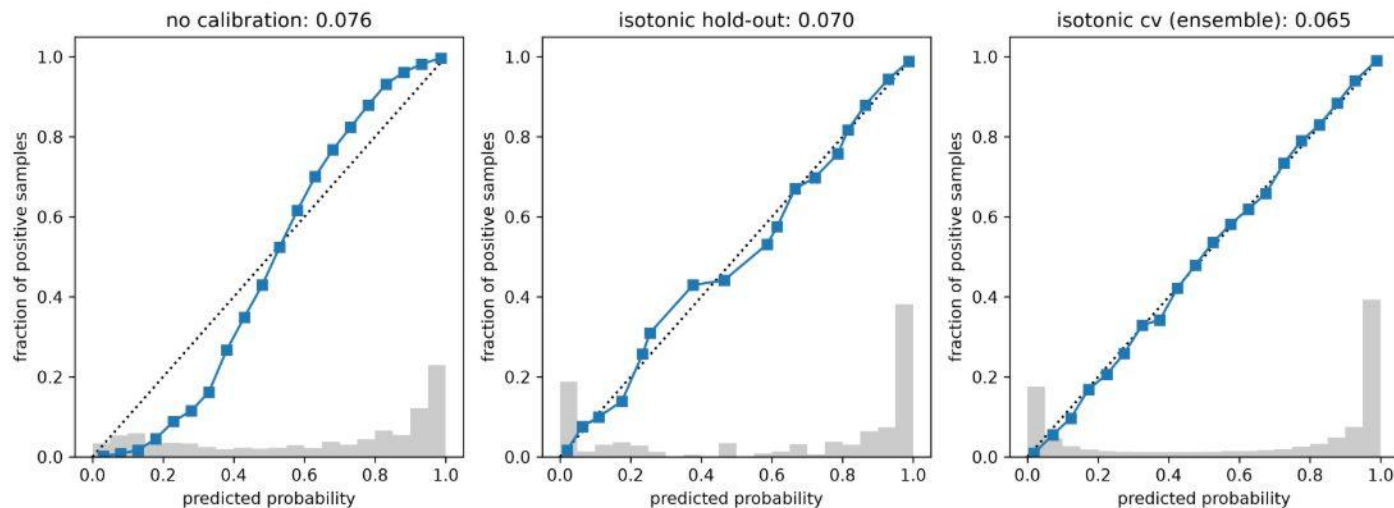
```
from sklearn.frozen import FrozenEstimator
# the forest is already fitted: freeze it, fit only the calibrator
cal_rf = CalibratedClassifierCV(FrozenEstimator(rf), method='sigmoid')
cal_rf.fit(X_val, y_val)
scores_sigm = cal_rf.predict_proba(X_test)[:, 1]

cal_rf_iso = CalibratedClassifierCV(FrozenEstimator(rf), method='isotonic')
cal_rf_iso.fit(X_val, y_val)
scores_iso = cal_rf_iso.predict_proba(X_test)[:, 1]
```



Cross-validated Calibration

```
cal_rf_iso_cv = CalibratedClassifierCV(RandomForestClassifier(random_
                                     method='isotonic', cv=5)
cal_rf_iso_cv.fit(X_train, y_train)
scores_iso_cv = cal_rf_iso_cv.predict_proba(X_test)[: , 1]
```



Logging competing models to MLflow

```
import mlflow
mlflow.setexperiment("week07-evaluation")
for name, model in candidates.items():
    with mlflow.start_run(run_name=name):
        model.fit(X_train, y_train)
        p = model.predict_proba(X_test)[: , 1]
        mlflow.log_metrics({"brier": brier_score_loss(y_test, p),
                           "log_loss": log_loss(y_test, p),
                           "roc_auc": roc_auc_score(y_test, p)})
        fig, ax = plt.subplots()
        CalibrationDisplay.from_predictions(y_test, p, ax=ax, name=name)
        mlflow.log_figure(fig, f"calibration{name}.png")
```

- Lab section 8: five models on the Adult census data; compare runs in `uv run mlflow ui`

Recap on imbalanced data

Two sources of imbalance

- Asymmetric cost
- Asymmetric data

Mammography Data, baselines

```
from sklearn.model_selection import cross_validate
from sklearn.linear_model import LogisticRegression

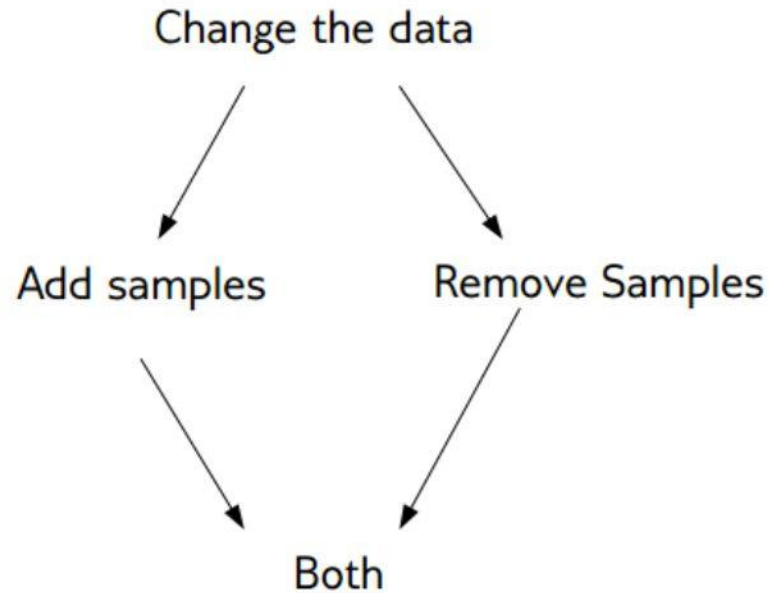
scores = cross_validate(LogisticRegression(),
                        X_train, y_train, cv=10,
                        scoring=('roc_auc', 'average_precision'))
scores['test_roc_auc'].mean(), scores['test_average_precision'].mean()
```

0.920, 0.635

```
from sklearn.ensemble import RandomForestClassifier
scores = cross_validate(RandomForestClassifier(random_state=0),
                        X_train, y_train, cv=10,
                        scoring=('roc_auc', 'average_precision'))
scores['test_roc_auc'].mean(), scores['test_average_precision'].mean()
```

0.936, 0.732

Basic Approaches



Change the training procedure

Random Undersampling

```
from imblearn.pipeline import make_pipeline as make_imb_pipeline

undersample_pipe = make_imb_pipeline(RandomUnderSampler(), LogisticRegression())
scores = cross_validate(undersample_pipe,
                        X_train, y_train, cv=10,
                        scoring=('roc_auc', 'average_precision'))
scores['test_roc_auc'].mean(), scores['test_average_precision'].mean()
# baseline was 0.920, 0.635
```

0.918, 0.535

```
undersample_pipe_rf = make_imb_pipeline(RandomUnderSampler(),
                                         RandomForestClassifier())
scores = cross_validate(undersample_pipe_rf,
                        X_train, y_train, cv=10,
                        scoring=('roc_auc', 'average_precision'))
scores['test_roc_auc'].mean(), scores['test_average_precision'].mean()
# baseline was 0.936, 0.732
```

0.949, 0.591

Class-weights

- Instead of repeating samples, re-weight the loss function.
- Works for most models!
- Same effect as over-sampling (though not random), but not as expensive (dataset size the same).

Using Class-Weights

```
scores = cross_validate(LogisticRegression(class_weight='balanced'),  
                        X_train, y_train, cv=10,  
                        scoring=('roc_auc', 'average_precision'))  
scores['test_roc_auc'].mean(), scores['test_average_precision'].mean()  
# baseline was 0.920, 0.635
```

0.919, 0.582

```
scores = cross_validate(RandomForestClassifier(class_weight='balanced'),  
                        X_train, y_train, cv=10,  
                        scoring=('roc_auc', 'average_precision'))  
scores['test_roc_auc'].mean(), scores['test_average_precision'].mean()  
# baseline was 0.936, 0.732
```

0.938, 0.725

What goes wrong in practice

```
# resampling BEFORE cross-validation
X_res, y_res = SMOTE().fit_resample(X_train, y_train)
cross_validate(RandomForestClassifier(), X_res, y_res, cv=5, scoring="average_precision")
```

reported average precision: 0.989 # honest (SMOTE inside the pipeline): 0.688

- Resampling before splitting leaks: synthetic points built from validation positives, balanced validation folds
- Resampling and class weights **distort probabilities**: re-calibrate, or tune the threshold afterwards
- For well-specified probabilistic models, "fixing" imbalance can hurt: van den Goorbergh et al., *JAMIA* 2022; Elor & Averbuch-Elor, *To SMOTE, or not to SMOTE?* 2022

Try it 3: spot the leak (10 min)

Lab section 6. A colleague reports a big gain from SMOTE.

1. Run their code and note the reported average precision
2. Find the problem and fix it (hint: `imblearn.pipeline`)
3. Which number would you put in a paper? What else would you report?

Summary

- Pick the metric from the application; report a ranking metric and a thresholded one
- The threshold is a decision: tune it with CV (`TunedThresholdClassifierCV`), with costs if you have them
- Check calibration (reliability diagram, Brier score); calibrate with `CalibratedClassifierCV`
- Imbalance: calibrate and tune the threshold first; class weights are cheap; resampling is an experiment
- Resample inside the pipeline, never before the split

Backup

The Zoo

		True condition			
Total population		Condition positive	Condition negative	Prevalence = $\frac{\Sigma \text{Condition positive}}{\Sigma \text{Total population}}$	Accuracy (ACC) = $\frac{\Sigma \text{True positive} + \Sigma \text{True negative}}{\Sigma \text{Total population}}$
Predicted condition	Predicted condition positive	True positive , Power	False positive , Type I error	Positive predictive value (PPV), Precision = $\frac{\Sigma \text{True positive}}{\Sigma \text{Predicted condition positive}}$	False discovery rate (FDR) = $\frac{\Sigma \text{False positive}}{\Sigma \text{Predicted condition positive}}$
	Predicted condition negative	False negative , Type II error	True negative	False omission rate (FOR) = $\frac{\Sigma \text{False negative}}{\Sigma \text{Predicted condition negative}}$	Negative predictive value (NPV) = $\frac{\Sigma \text{True negative}}{\Sigma \text{Predicted condition negative}}$
		True positive rate (TPR), Recall, Sensitivity, probability of detection = $\frac{\Sigma \text{True positive}}{\Sigma \text{Condition positive}}$	False positive rate (FPR), Fall-out, probability of false alarm = $\frac{\Sigma \text{False positive}}{\Sigma \text{Condition negative}}$	Positive likelihood ratio (LR+) = $\frac{\text{TPR}}{\text{FPR}}$	Diagnostic odds ratio (DOR) = $\frac{\text{LR+}}{\text{LR-}}$ $F_1 \text{ score} = \frac{2}{\frac{1}{\text{Recall}} + \frac{1}{\text{Precision}}}$
		False negative rate (FNR), Miss rate = $\frac{\Sigma \text{False negative}}{\Sigma \text{Condition positive}}$	True negative rate (TNR), Specificity (SPC) = $\frac{\Sigma \text{True negative}}{\Sigma \text{Condition negative}}$	Negative likelihood ratio (LR-) = $\frac{\text{FNR}}{\text{TNR}}$	

https://en.wikipedia.org/wiki/Precision_and_recall

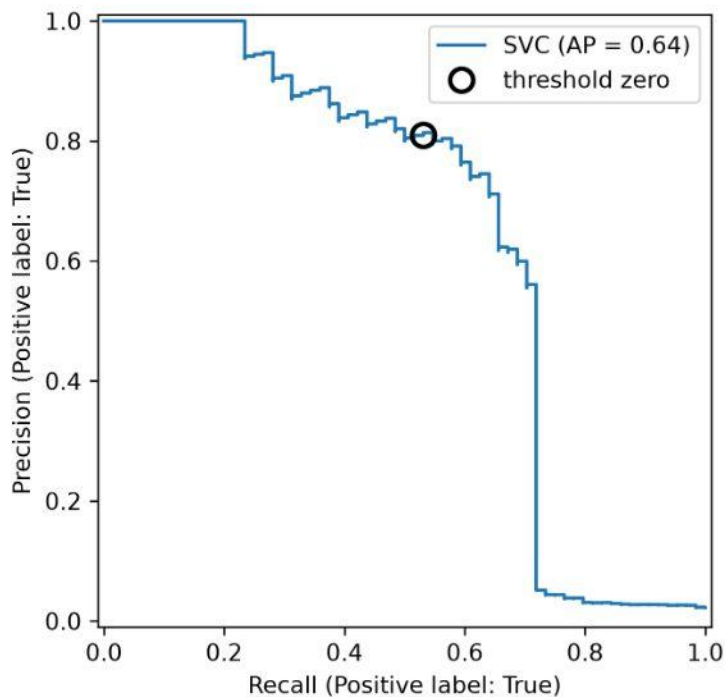
Side note: Scikit-learn plotting API

```
pr_svc = PrecisionRecallDisplay.from_estimator(svc, X_test, y_test, name='SVC')  
# plot pops up here  
vars(pr_svc)
```

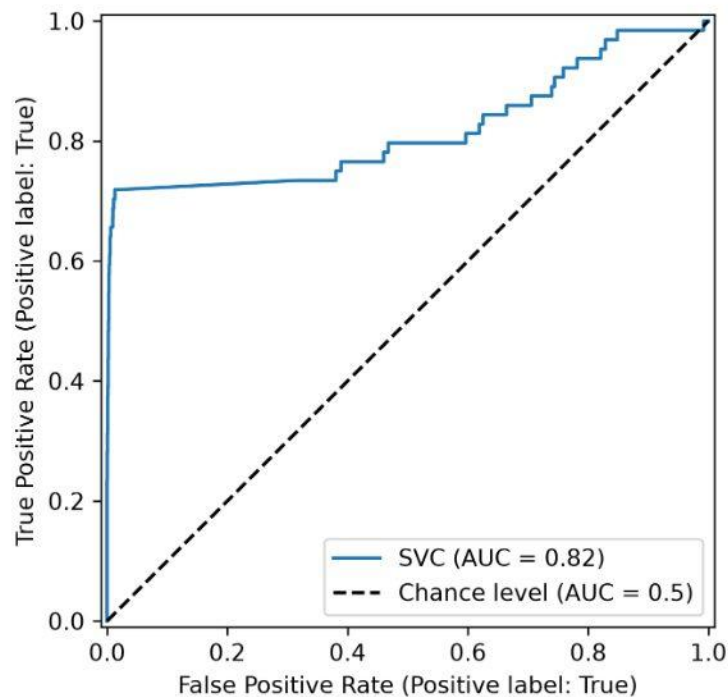
```
{'precision': array([0.023, 0.023, ..., 1.    , 1.    ]),  
'recall': array([1.    , 0.984, ..., 0.016, 0.    ]),  
'average_precision': 0.635,  
'estimator_name': 'SVC',  
'line_': <matplotlib.lines.Line2D at ... >,  
'ax_': <Axes: ...>,  
'figure_': <Figure size 640x480 with 1 Axes>}
```

```
# plot again (e.g. into another axes) without recomputing  
pr_svc.plot(ax=ax)  
# or start from scores you already have  
PrecisionRecallDisplay.from_predictions(y_test, svc.decision_function(X_test))
```

PR-Curve



ROC-Curve



- Share one axis (recall = TPR), though it is on different sides
- Linear interpolation is meaningful on the ROC curve but not on the PR curve

Further reading on evaluation curves

- [The Relationship Between Precision-Recall and ROC Curves](#) (Davis & Goadrich, ICML 2006)
- [Precision-Recall-Gain Curves: PR Analysis Done Right](#) (Flach & Kull, NeurIPS 2015)
- [Scikit-learn user guide: tuning the decision threshold](#)

Multi-class classification

Multi-class ROC AUC

- Hand & Till, 2001, one vs one

$$\frac{1}{c(c-1)} \sum_{j=1}^c \sum_{k \neq j}^c \text{AUC}(j, k)$$

- Provost & Domingos, 2000, one vs rest

$$\frac{1}{c} \sum_{j=1}^c p(j) \text{AUC}(j, \text{rest}_j)$$

```
roc_auc_score(y_test, lr.predict_proba(X_test), multi_class="ovo", average="macro")  
# scorer names: roc_auc_ovo, roc_auc_ovo_weighted, roc_auc_ovr, roc_auc_ovr_weighted
```

Built-in scoring

```
from sklearn.metrics import get_scorer_names
print(get_scorer_names())
```

accuracy	adjusted_mutual_info_score	adjusted_rand_score
average_precision	balanced_accuracy	completeness_score
d2_absolute_error_score	d2_brier_score	d2_log_loss_score
explained_variance	f1	f1_macro
f1_micro	f1_samples	f1_weighted
fowlkes_mallows_score	homogeneity_score	jaccard
jaccard_macro	jaccard_micro	jaccard_samples
jaccard_weighted	matthews_corrcoef	mutual_info_score
neg_brier_score	neg_log_loss	neg_max_error
neg_mean_absolute_error	neg_mean_absolute_percentage_error	
neg_mean_gamma_deviance	neg_mean_poisson_deviance	neg_mean_squared_error
neg_mean_squared_log_error	neg_median_absolute_error	neg_negative_likelihood_ratio
neg_root_mean_squared_error	neg_root_mean_squared_log_error	normalized_mutual_info_score
positive_likelihood_ratio	precision ... recall ...	r2 rand_score
roc_auc roc_auc_ovo roc_auc_ovo_weighted roc_auc_ovr roc_auc_ovr_weighted		
top_k_accuracy	v_measure_score	

The Scorer interface (vs the metrics interface)

```
# metric function interface: (y_true, y_pred or y_score)
y_probs = rf.predict_proba(X_test)
ap_rf = average_precision_score(y_test, y_probs[:, 1])
y_pred = rf.predict(X_test)
ba_rf = balanced_accuracy_score(y_test, y_pred)
# scorer interface: (estimator, X, y)
from sklearn.metrics import get_scorer, make_scorer
ap_scorer = get_scorer('average_precision')
ap_rf = ap_scorer(rf, X_test, y_test)
ba_scorer = get_scorer('balanced_accuracy')
ba_rf = ba_scorer(rf, X_test, y_test)
# turn a metric into a scorer
f2_scorer = make_scorer(fbeta_score, beta=2)
```

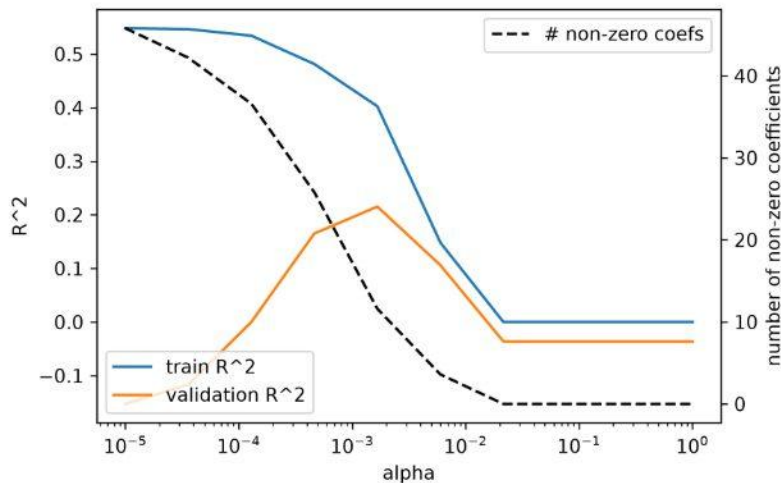
Providing your own callable

- Takes estimator, X, y
- Returns score – higher is better (always!)

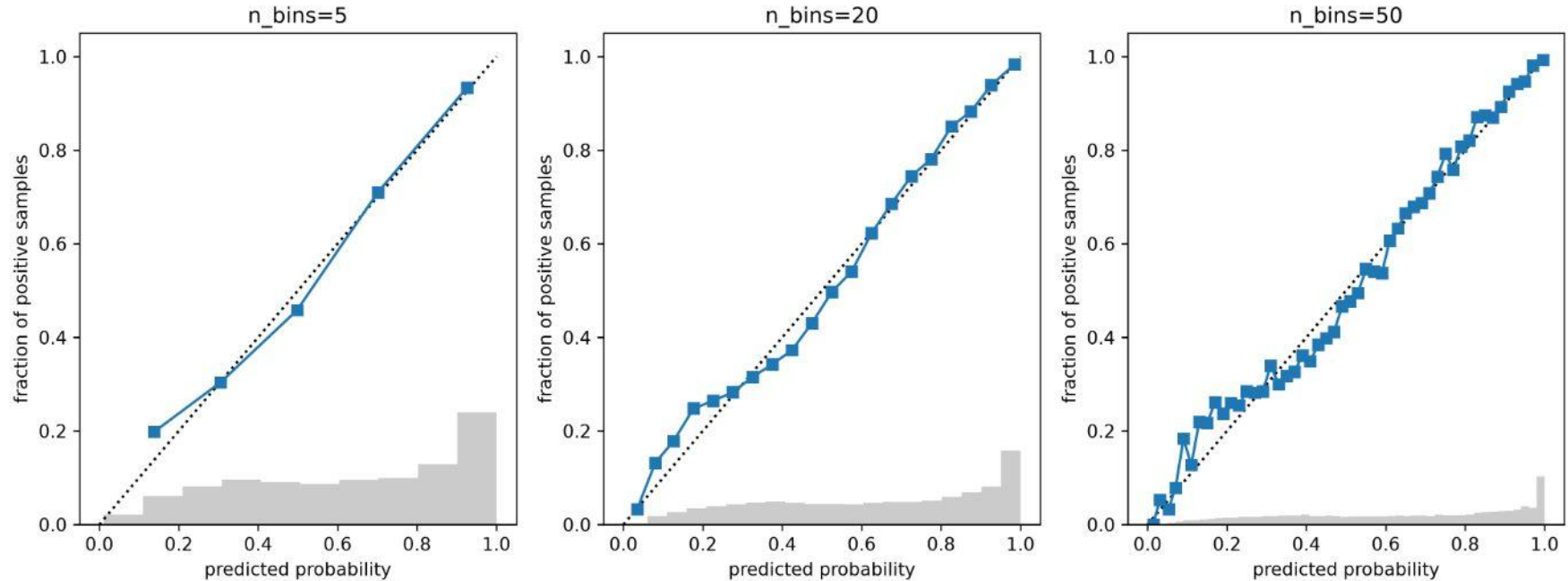
```
def accuracy_scoring(est, X, y):  
    return (est.predict(X) == y).mean()
```

You can access the model!

```
def nonzero(est, X, y):  
    return np.sum(est.coef_ != 0)  
  
tri = fetch_openml("triazines", version=1, as_frame=False)  
X_train, X_test, y_train, y_test = train_test_split(tri.data, tri.target, random_state=0)  
param_grid = {'alpha': np.logspace(-5, 0, 10)}  
# scoring can be string, single scorer, list or dict  
grid = GridSearchCV(Lasso(max_iter=10_000), param_grid, return_train_score=True,  
                    scoring={'r2': 'r2', 'num_nonzero': nonzero}, refit='r2')  
grid.fit(X_train, y_train)  
results = pd.DataFrame(grid.cv_results_)
```



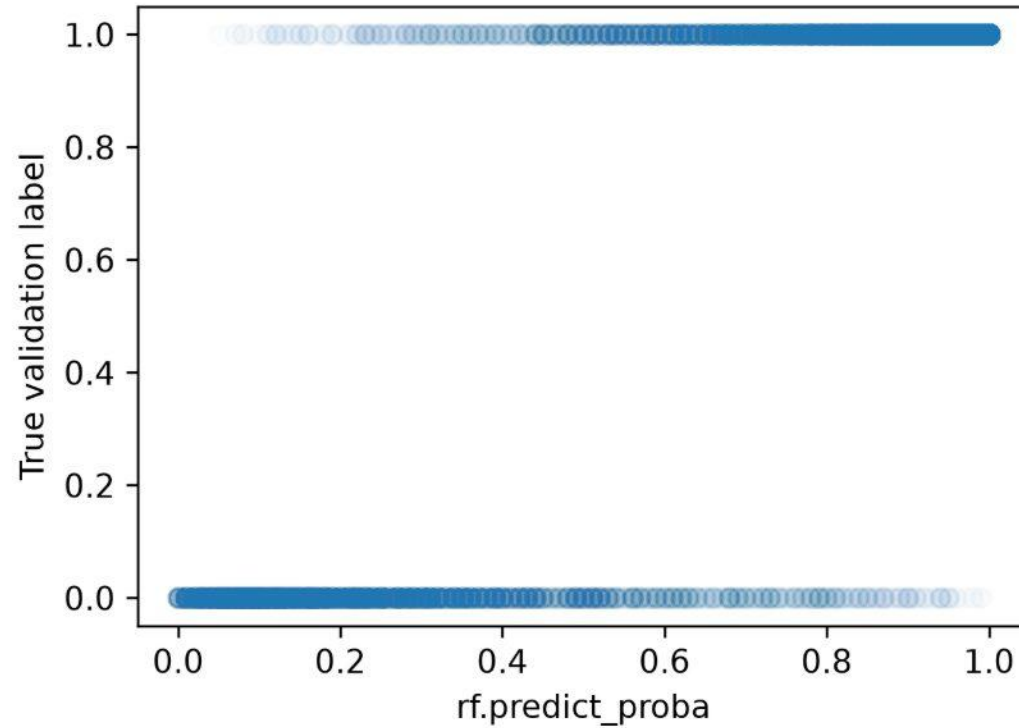
Influence of number of bins



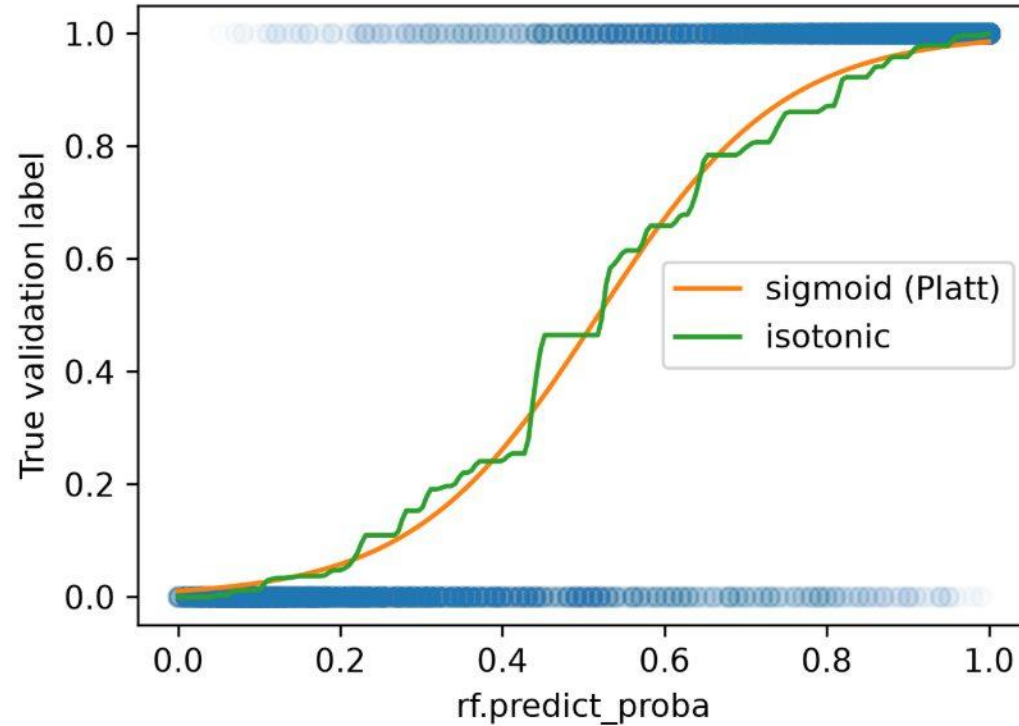
Building the model

- Using the training set is bad
- Either use hold-out set or cross-validation
- Cross-validation can be used to make unbiased probability predictions, use that as training set.

Fitting the calibration model



Fitting the calibration model



ensemble=True vs ensemble=False

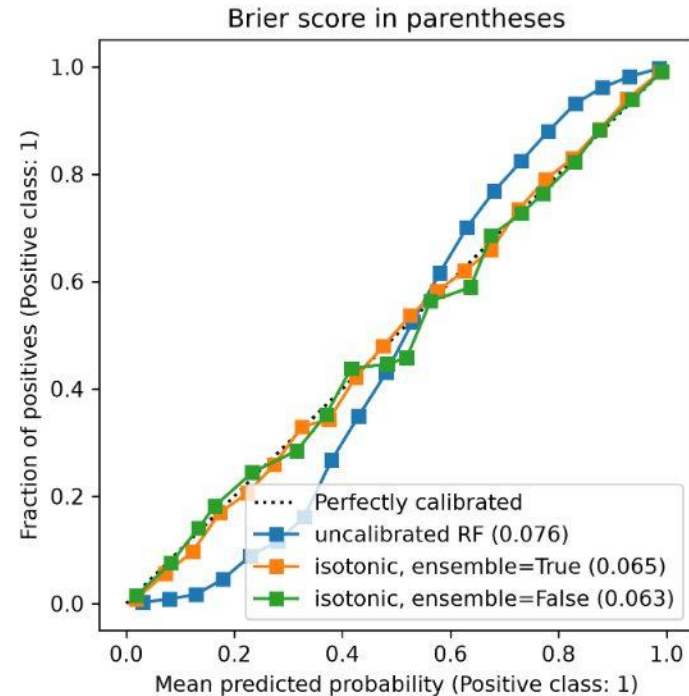
```
CalibratedClassifierCV(rf, method="isotonic",  
                      cv=5, ensemble=True)
```

5 (model, calibrator) pairs
probabilities averaged

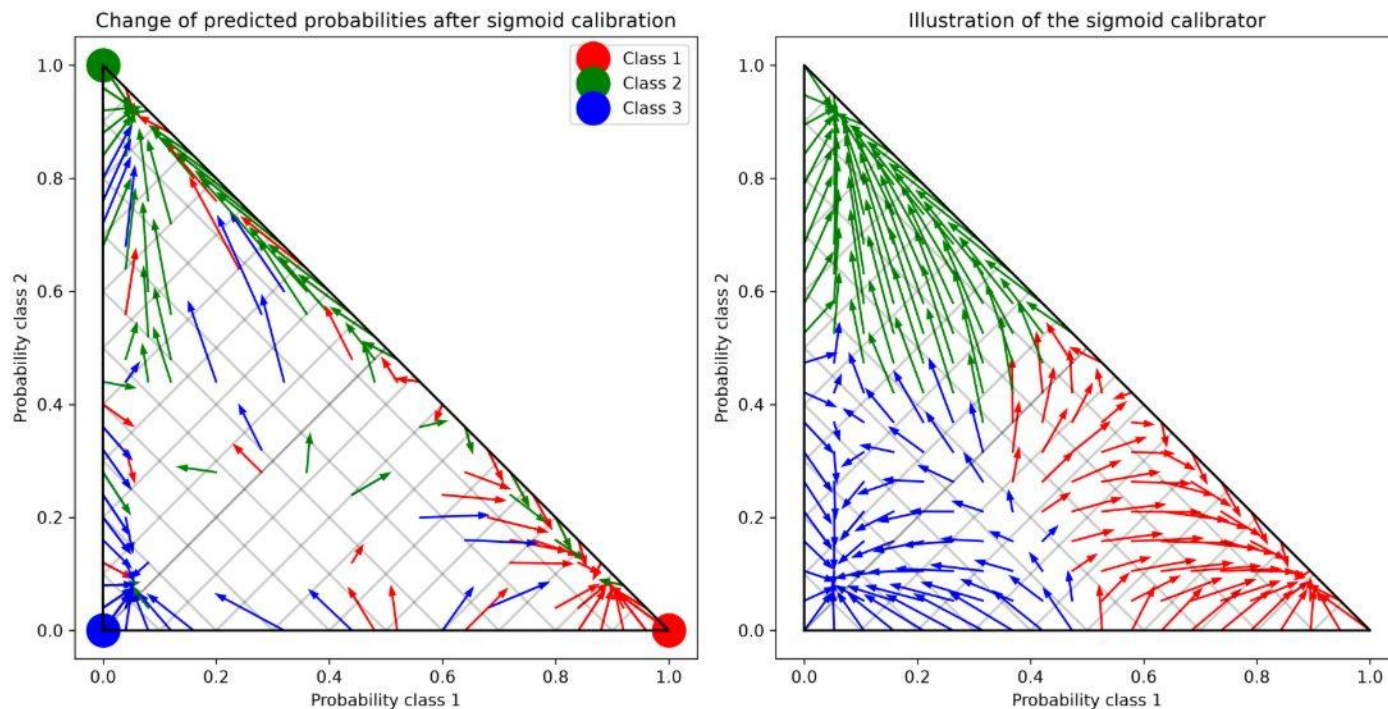
```
CalibratedClassifierCV(rf, method="isotonic",  
                      cv=5, ensemble=False)
```

calibrator fit on cross_val
output; one model refit

- `ensemble="auto"` (default): `False` for `alpha`
`FrozenEstimator`, `True` otherwise



Multi-Class Calibration



Log loss: uncalibrated RF 1.533 · sigmoid (one-vs-rest) 0.539 · temperature scaling 0.652

Why do we care?

- Why should cost be symmetric?
- All data is imbalanced
- Detect rare events

Changing Thresholds

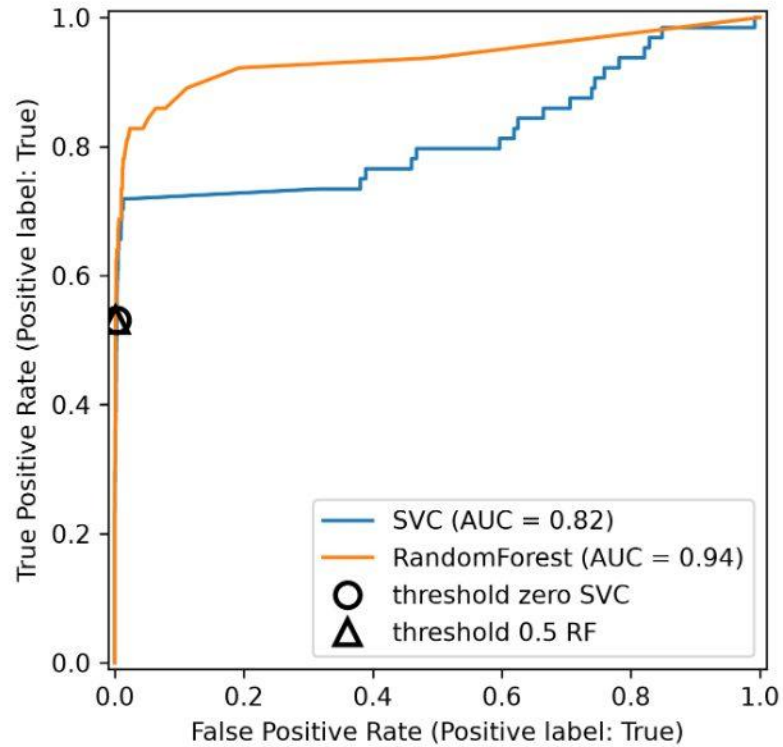
```
data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, stratify=data.target, random_state=0)
lr = make_pipeline(StandardScaler(), LogisticRegression()).fit(X_train, y_train)
y_pred = lr.predict(X_test)
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.94	0.94	0.94	53
1	0.97	0.97	0.97	90
accuracy			0.96	143

```
y_pred = lr.predict_proba(X_test)[:, 1] > .85
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.89	0.96	0.93	53
1	0.98	0.93	0.95	90
accuracy			0.94	143

ROC Curve



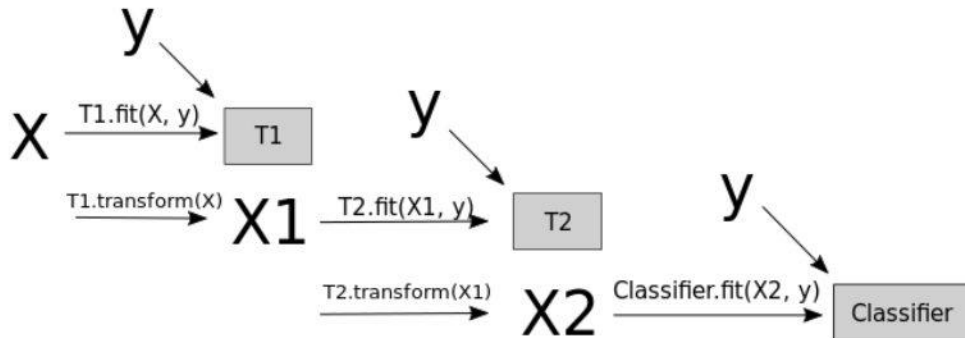
Remedies for the model

Scikit-learn vs resampling

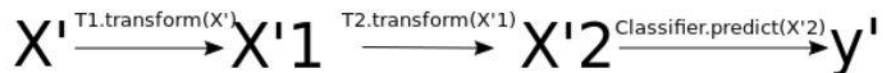
```
pipe = make_pipeline(T1(), T2(), Classifier())
```



```
pipe.fit(X, y)
```



```
pipe.predict(X')
```



Imbalanced-learn

<https://imbalanced-learn.org>

```
uv add imbalanced-learn    # already in the course environment
```

Extends `sklearn` API

Sampler

To resample a data set, each sampler implements:

```
data_resampled, targets_resampled = obj.fit_resample(data, targets)
```

In Pipelines: Sampling only done in `fit`!

Random Undersampling

```
from imblearn.under_sampling import RandomUnderSampler
rus = RandomUnderSampler(replacement=False)
X_train_subsample, y_train_subsample = rus.fit_resample(
    X_train, y_train)
print(X_train.shape)
print(X_train_subsample.shape)
print(np.bincount(y_train_subsample))
```

```
(8387, 6)
(390, 6)
[195 195]
```

Random Oversampling

```
from imblearn.over_sampling import RandomOverSampler
ros = RandomOverSampler()
X_train_oversample, y_train_oversample = ros.fit_resample(
    X_train, y_train)
print(X_train.shape)
print(X_train_oversample.shape)
print(np.bincount(y_train_oversample))
```

```
(8387, 6)
(16384, 6)
[8192 8192]
```

Random Oversampling

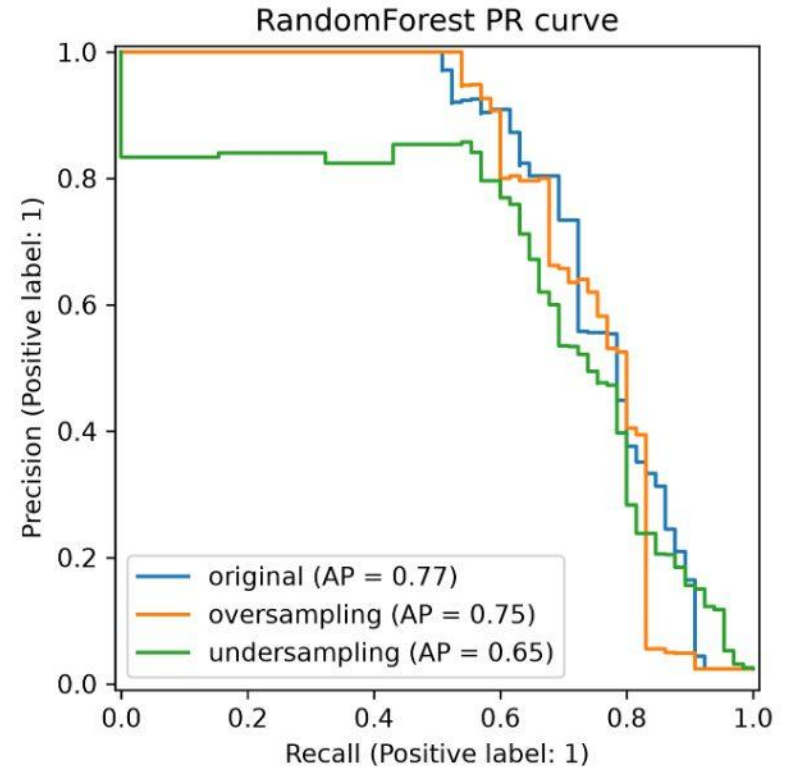
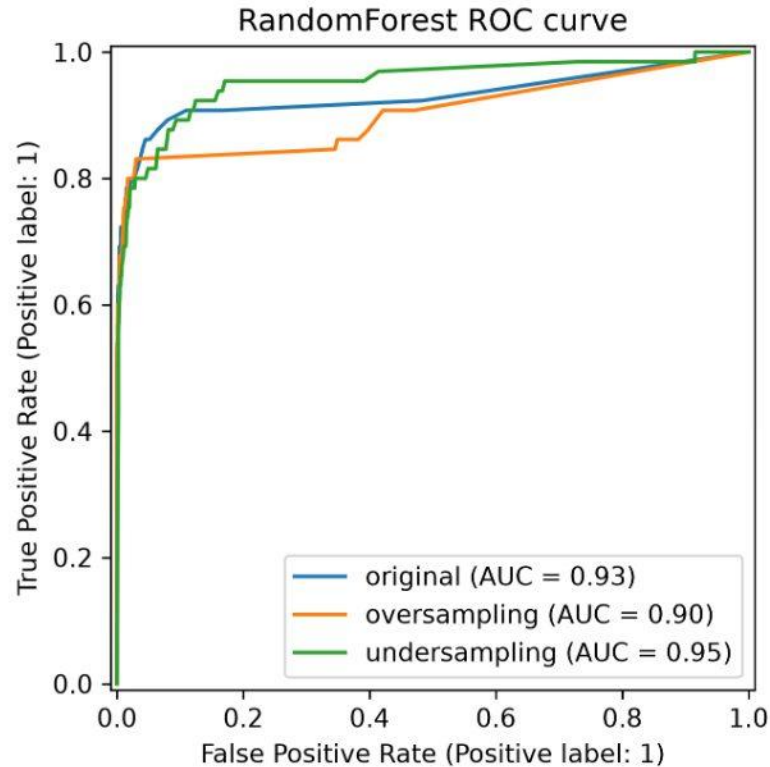
```
oversample_pipe = make_imb_pipeline(RandomOverSampler(), LogisticRegression())
scores = cross_validate(oversample_pipe,
                        X_train, y_train, cv=10,
                        scoring=('roc_auc', 'average_precision'))
scores['test_roc_auc'].mean(), scores['test_average_precision'].mean()
# baseline was 0.920, 0.635
```

0.918, 0.583

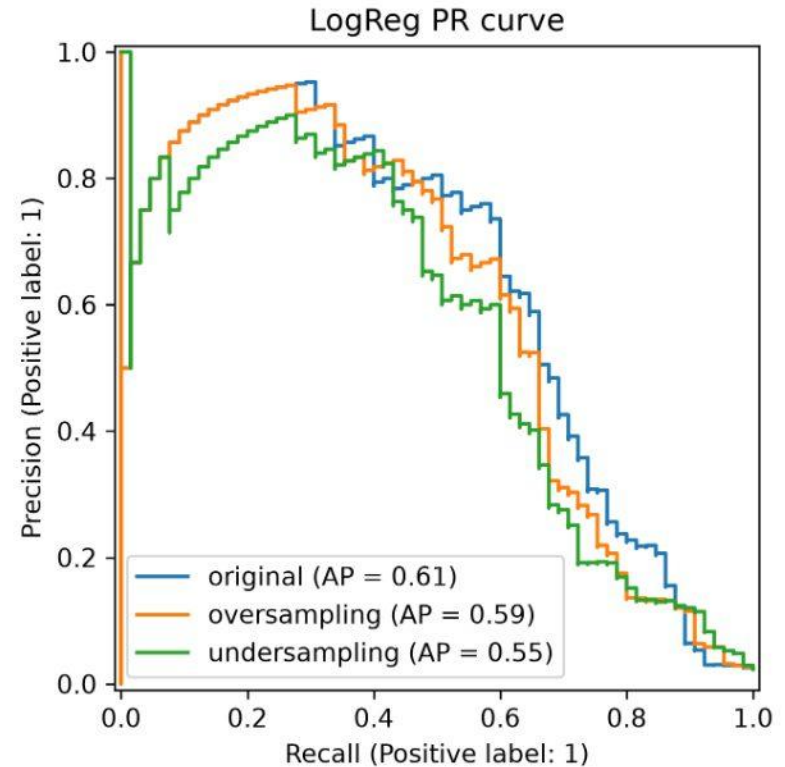
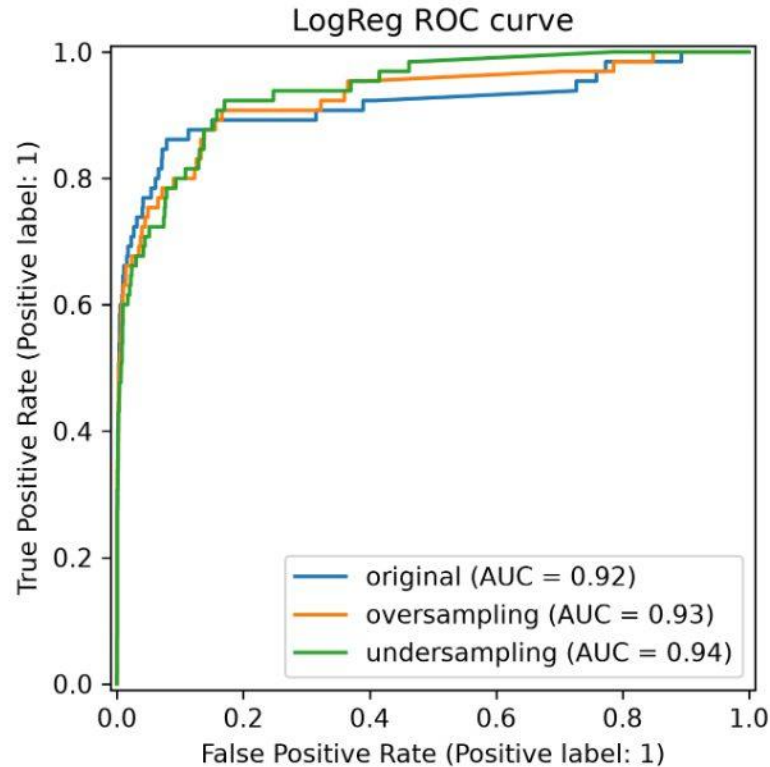
```
oversample_pipe_rf = make_imb_pipeline(RandomOverSampler(),
                                       RandomForestClassifier())
scores = cross_validate(oversample_pipe_rf,
                        X_train, y_train, cv=10,
                        scoring=('roc_auc', 'average_precision'))
scores['test_roc_auc'].mean(), scores['test_average_precision'].mean()
# baseline was 0.936, 0.732
```

0.919, 0.708

Curves for Random Forest



Curves for LogReg



Class-weights in linear models

$$\min_{w \in \mathbb{R}^p, b \in \mathbb{R}} C \sum_{i=1}^n \log(\exp(-y_i(w^T \mathbf{x}_i + b)) + 1) + ||w||_2^2$$

$$\min_{w \in \mathbb{R}^p, b \in \mathbb{R}} C \sum_{i=1}^n c_{y_i} \log(\exp(-y_i(w^T \mathbf{x}_i + b)) + 1) + ||w||_2^2$$

Similar for linear and non-linear SVM

Class weights in trees

Gini Index:

$$H_{\text{gini}}(X_m) = \sum_{k \in \square} p_{mk}(1 - p_{mk})$$

$$H_{\text{gini}}(X_m) = \sum_{k \in \square} c_k p_{mk}(1 - p_{mk})$$

Prediction:

Weighted vote

Ensemble Resampling

- Random resampling separate for each instance in an ensemble!
- Chen, Liaw, Breiman: “Using random forest to learn imbalanced data.” (2004)
- Liu, Wu, Zhou: “Exploratory Undersampling for Class-Imbalance Learning” (2009)
- Not in scikit-learn; available in imbalanced-learn

Easy Ensemble with imblearn

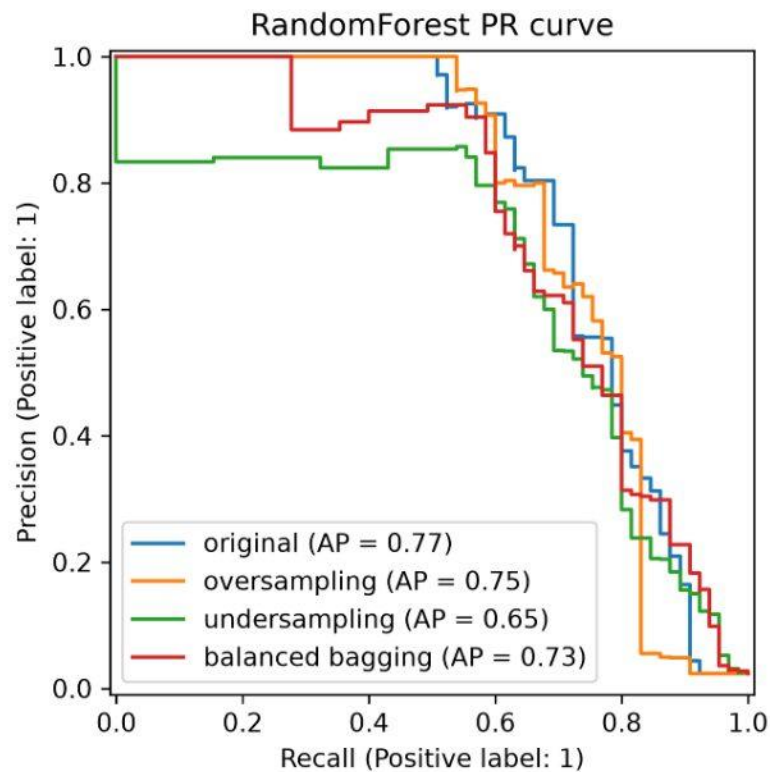
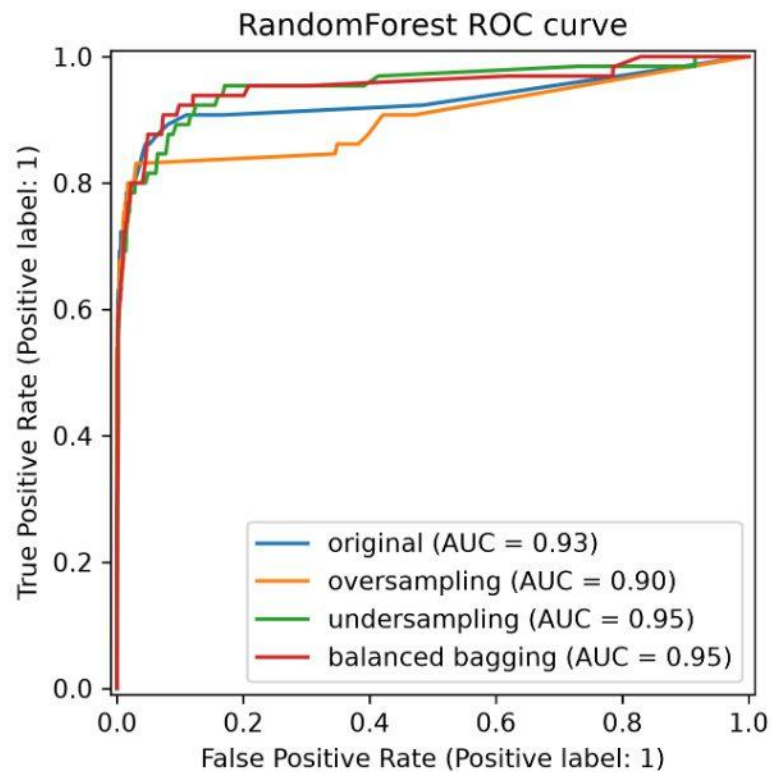
```
from sklearn.tree import DecisionTreeClassifier
from imblearn.ensemble import BalancedBaggingClassifier, BalancedRandomForestClassifier

tree = DecisionTreeClassifier(max_features='sqrt')
resampled_rf = BalancedBaggingClassifier(estimator=tree, n_estimators=100,
                                         random_state=0)

scores = cross_validate(resampled_rf,
                        X_train, y_train, cv=10,
                        scoring=('roc_auc', 'average_precision'))
scores['test_roc_auc'].mean(), scores['test_average_precision'].mean()
# baseline was 0.936, 0.732
```

0.954, 0.667

```
BalancedRandomForestClassifier(random_state=0) # 0.952, 0.658
```

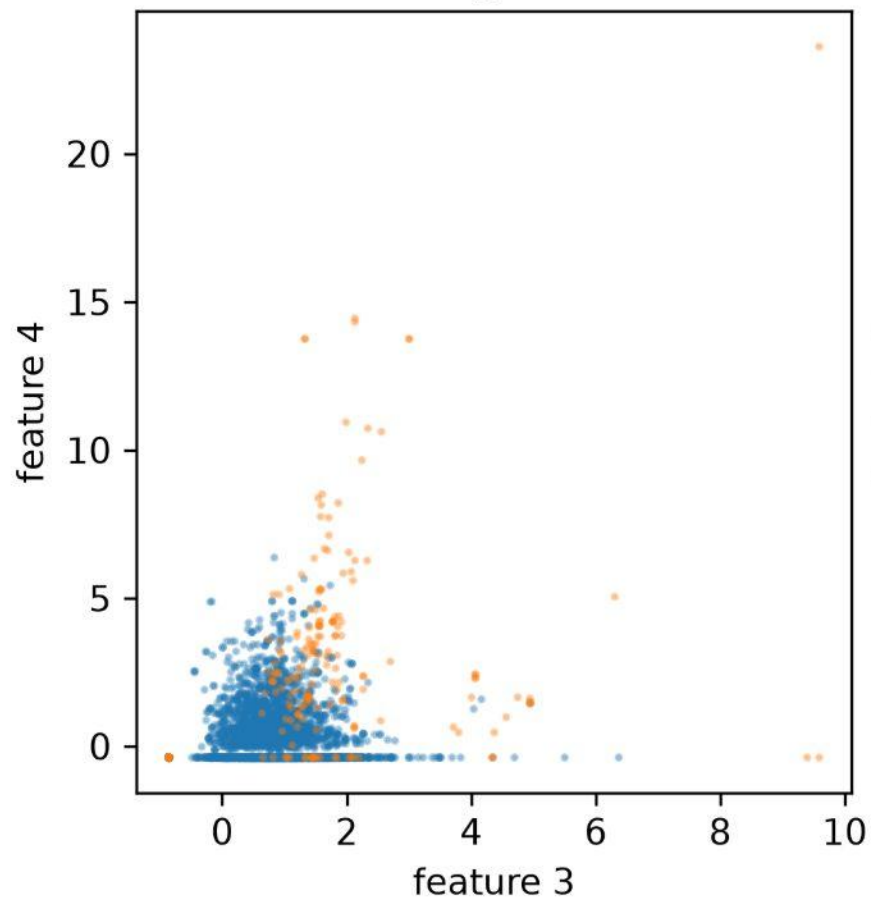


Synthetic Sample Generation

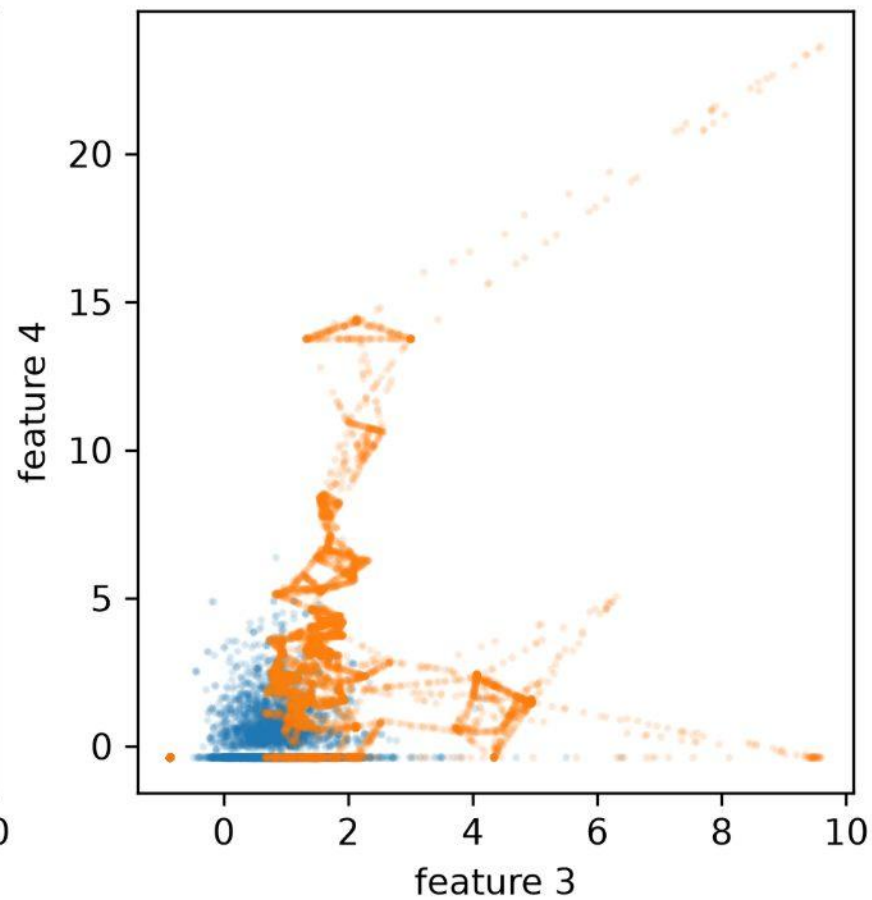
Synthetic Minority Oversampling Technique (SMOTE)

- Adds synthetic interpolated data to smaller class
- For each sample in minority class:
 - Pick random neighbor from k neighbors (of the same class).
 - Pick point on line connecting the two uniformly.
 - Repeat.

Original



SMOTE



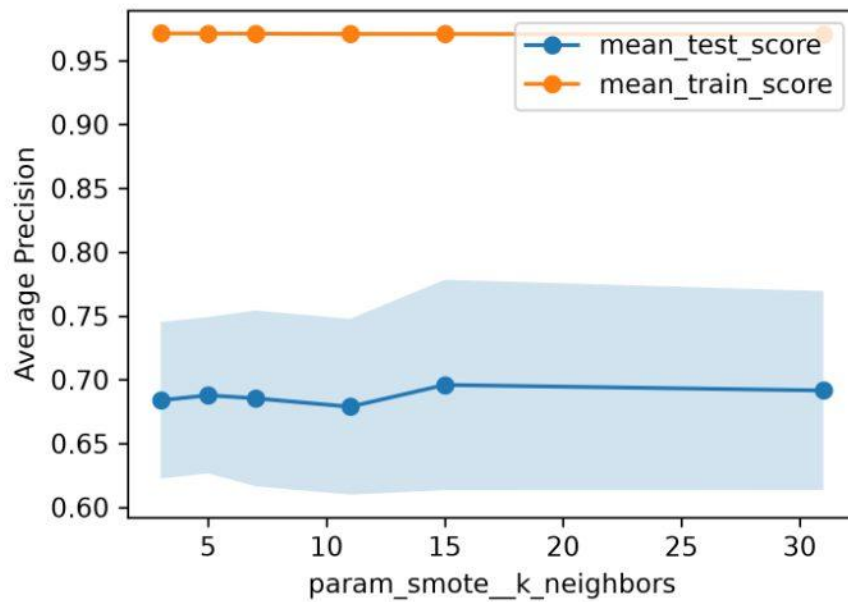
```
smote_pipe = make_imb_pipeline(SMOTE(), LogisticRegression())
scores = cross_validate(smote_pipe, X_train, y_train, cv=10,
                        scoring=('roc_auc', 'average_precision'))
pd.DataFrame(scores)[['test_roc_auc', 'test_average_precision']].mean()
# baseline was 0.920, 0.635
```

0.920, 0.584

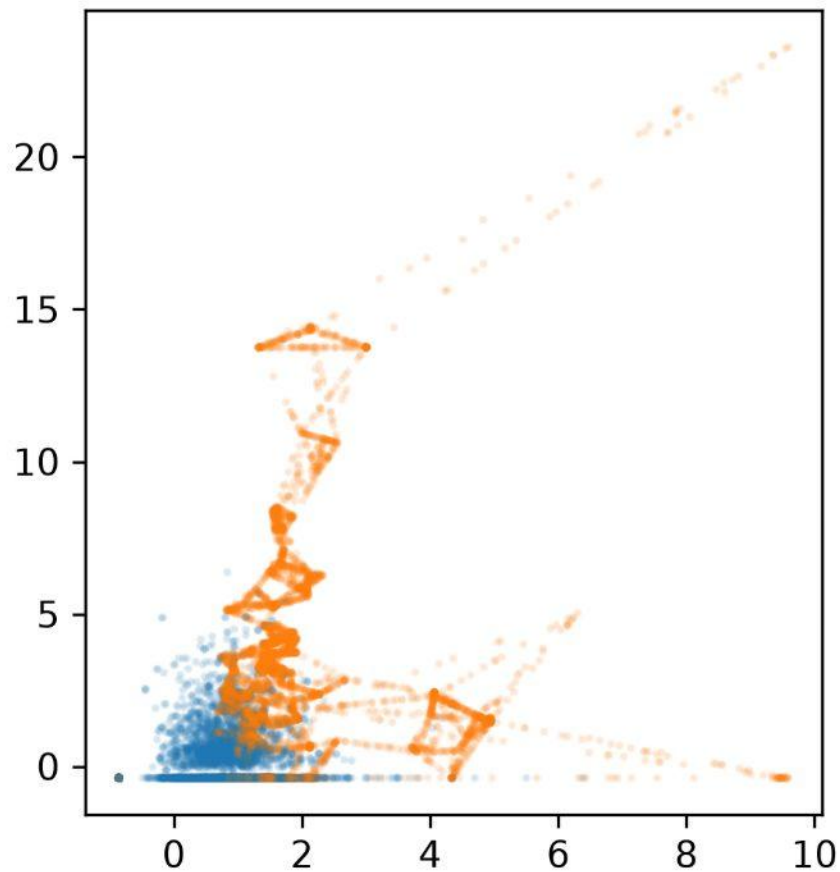
```
smote_pipe_rf = make_imb_pipeline(SMOTE(),
                                   RandomForestClassifier())
scores = cross_validate(smote_pipe_rf, X_train, y_train, cv=10,
                        scoring=('roc_auc', 'average_precision'))
pd.DataFrame(scores)[['test_roc_auc', 'test_average_precision']].mean()
# baseline was 0.936, 0.732
```

0.943, 0.690

```
param_grid = {'smote__k_neighbors': [3, 5, 7, 11, 15, 31]}
search = GridSearchCV(smote_pipe_rf, param_grid, cv=5,
                      scoring="average_precision", return_train_score=True)
search.fit(X_train, y_train)
results = pd.DataFrame(search.cv_results_)
results.plot("param_smote__k_neighbors", ["mean_test_score", "mean_train_score"])
```



SMOTE k_neighbors=5



SMOTE k_neighbors=15

